Reporte

Nombre: Roberto Carlos Cantu González	**Matrícula**: 2114360
Nombre: Ángel Daniel Jiménez Fuantos	**Matrícula**: 2117083
Nombre: Mario Ivan González Martínez	**Matrícula**: 2117170
Nombre del curso: Componentes de Lógica de Negocios	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: Actividad 3. Informe
Fecha: 15/10/2026	
Bibliografía (Formato APA):	

Objetivo:

En esta actividad convertirán el análisis de negocio elaborado en la Unidad 2 en una propuesta de arquitectura de software. Trabajarán con la misma empresa y el mismo caso. La inteligencia artificial podrá apoyar el diseño y la revisión, pero cada decisión deberá justificarse con evidencia del proyecto.

El objetivo es organizar las reglas y responsabilidades del sistema antes de programarlo por completo. Al terminar, el equipo deberá demostrar cómo pasó de los requerimientos de la Unidad 2 a una arquitectura que separa el dominio, la aplicación y la infraestructura.

La actividad no consiste en pedir a una herramienta de IA que produzca una solución terminada. El trabajo consiste en plantear decisiones, solicitar alternativas, revisar las respuestas y documentar por qué una propuesta sí corresponde o no al negocio analizado.

Etapa 1. Preparar el caso

Para el diseño arquitectural de TecnoStock, se toma como insumo el análisis de requerimientos desarrollado en la Unidad 2. Se delimita el alcance al Proceso Principal Operativo, el cual articula la consulta de existencias globales, la atención de ventas en mostrador con identificación por número de serie, la reserva de inventario por apartados, las transferencias inter-sucursales y la atención de devoluciones bajo la supervisión de un rol autorizado.

Se seleccionaron 5 casos de uso representativos del proceso principal que abarcan flujos normales, validaciones estrictas y reglas con excepción/autorización:

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Procesar Venta Directa con Serie	Cajero / Vendedor	Escaneo obligatorio de Número de Serie/IMEI individual. Cobro multicanal.	Descuento de stock en tiempo real, cambio de estado de la serie a Vendida y emisión del ticket fiscal.
CU-02	Registrar Orden de Apartado	Vendedor / Cajero	Exigencia de anticipo mínimo del 20%. Plazo límite de 30 días naturales. Máximo 3 apartados activos por cliente.	Generación de comprobante de apartado y bloqueo lógico inmediato de la unidad a estado Reservado.
CU-03	Procesar Vencimiento de Apartados	Sistema (Proceso Nocturno)	Verificación de plazo acumulado > 30 días sin liquidación. Retención del 10% por penalización.	Liberación del ítem a estado Disponible y generación de saldo a favor del cliente.
CU-04	Transferir Mercancía Inter-Sucursal	Vendedor / Almacén	Confirmación de existencia local en otra sucursal. Doble validación de número de serie en origen y destino.	Creación de orden de traslado, asignación de estado En Tránsito y recepción confirmada en destino.

CU-05	Procesar Devolución Especial	Cajero / Supervisor	Límite máximo de 15 días posteriores a la venta. Verificación de empaque/accesorios y autenticación obligatoria con clave del Supervisor.	Generación de nota de crédito/reembolso y reingreso de la serie al inventario activo.

Etapa 2. Elegir el patrón de lógica de negocio

Para seleccionar la arquitectura de lógica de negocio más adecuada para TecnoStock, se compararon los tres patrones fundamentales propuestos (Transaction Script, Table Module y Domain Model) con base en las exigencias reales del proyecto:

Criterio del caso	Transaction Script	Table Module	Domain Model (Seleccionado)	Evidencia de la U2
Complejidad de reglas	Lógica lineal amarrada al script de transacción. Dificulta mantener reglas cruzadas.	Lógica organizada alrededor de tablas de base de datos. Adecuado solo para operaciones CRUD simples.	Encapsula estado y comportamiento en entidades y agregados ricos. Permite aislar reglas complejas.	Regla del 20% de anticipo mínimo, temporizador de 30 días y penalización del 10% en apartados.
Manejo de estados e invariantes	Sin garantía de invariantes; las validaciones deben repetirse en cada procedimiento.	Depende de filas de datos. No garantiza que un objeto mantenga un estado válido en memoria.	Garantiza que las entidades cambien de estado solo si se cumplen las reglas (invariantes protegidas).	El ítem apartado no puede agregarse a otra venta (bloqueo lógico inmediato a nivel de objeto).
Trazabilidad de números de serie	Riesgo de saltar el escaneo o procesar series duplicadas.	Maneja las series como simples cadenas de texto en celdas de una tabla.	Asigna identidad única y ciclo de vida al objeto ItemSerie con estados estrictos.	Obligatoriedad del escaneo de código de barras de serie en ventas, recepciones y transferencias.

Interacción entre conceptos	Código sumamente duplicado entre sucursales y terminales de cobro.	Dificultad para coordinar múltiples tablas en transacciones complejas.	Facilita la coordinación de Aggregates mediante Application Services y Domain Events.	Sincronización del inventario global en tiempo real a través de las 5 sucursales.

Se ha tomado la decisión arquitectural de adoptar Domain Model como el patrón principal para el núcleo de la solución de TecnoStock.

Esta elección se respalda con cuatro evidencias clave de la Unidad 2:

1. Protección de Invariantes Complejas (Necesidad N-02 / Regla de Apartados): Un apartado exige un anticipo mínimo del 20% y una vigencia de 30 días. El modelo de dominio garantiza que el objeto Apartado no pueda instanciarse o guardarse en un estado inválido si no satisface la regla del 20%.
2. Ciclo de Vida e Identidad de Números de Serie (Necesidad N-03): La exigencia de rastrear computadoras y celulares por número de serie/IMEI requiere que cada unidad mantenga su propia identidad y transiciones de estado explícitas. En un *Domain Model*, la entidad ItemSerie expone métodos con significado de negocio como BloquearParaReserva() o MarcarComoVendido(), impidiendo cambios de estado arbitrarios.
3. Erradicación de Ventas Accidentales de Productos Reservados (Criterio de Éxito SCR-01): La principal falla del sistema anterior era la venta accidental de productos apartados. Con *Domain Model*, el bloqueo lógico se ejecuta directamente en la entidad del dominio en el momento en que se procesa el anticipo, haciendo imposible que el catálogo para venta inmediata contabilice esa unidad.
4. Estandarización de Autorizaciones (Restricción CON-02 / Devoluciones): Las devoluciones requieren la validación de condiciones físicas y la autenticación explícita del Supervisor. Encapsular este flujo en el dominio asegura que la lógica de autorización no pueda ser omitida o bypasseadas desde la interfaz gráfica o puntos de cobro alternativos.

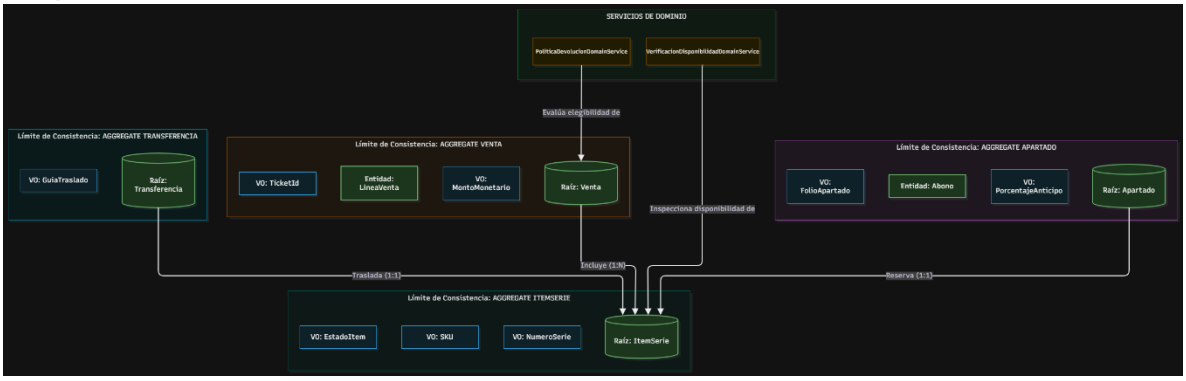
Etapa 3. Construir el mapa de dominio

A continuación se resume la clasificación de todos los elementos del dominio para TecnoStock atendiendo a las preguntas guía:

Elemento	Pregunta que debe responder el equipo	Aplicación y Respuesta Concreta en el Caso TecnoStock
Lenguaje Ubicuo	Términos usados por usuarios, analistas y desarrolladores con el mismo significado.	Vocabulario estandarizado: ItemSerie, Apartado, Anticipo, Reserva Lógica, Penalización por Vencimiento, Transferencia Inter-Sucursal y Autorización Supervisor.
Entidades	Conceptos con identidad y ciclo de vida que deben distinguirse aun cuando cambien sus datos.	**Abono**: Registro individual de un pago parcial asociado a un apartado con fecha y monto. **LineaVenta**: Detalle que vincula los artículos adquiridos dentro de una transacción.
Value Objects	Conceptos definidos por sus valores y reglas, sin identidad propia relevante.	Estructuras inmutables: **NumeroSerie** (formato alfanumérico o IMEI de 15 dígitos), **SKU** (código de catálogo), **MontoMonetario** (monto y moneda) y **PorcentajeAnticipo** (validación $\geq 20\%$).
Aggregates	Grupos que requieren un límite de consistencia y una raíz para controlar cambios.	**Apartado** (Raíz: gestiona vigencia de 30 días, anticipo y abonos). **ItemSerie** (Raíz: gestiona estados Disponible, Reservado, EnTransito, Vendido). **Venta** (Raíz: gestiona la transacción comercial y pagos combinados). **Transferencia** (Raíz: coordina el traslado entre sucursales).
Domain Services	Comportamientos del dominio que no pertenecen naturalmente a una sola entidad.	**PoliticaDevolucionDomainService**: Evalúa elegibilidad por tiempo (≤ 15 días) y exige firma de Supervisor.

		VerificacionDisponibilidadDomainService: Inspecciona existencias en las 5 tiendas.
Domain Events	Hechos del negocio que ya ocurrieron y pueden interesar a otros componentes.	Eventos emitidos: **ApartadoCreadoEvent**, **SerieBloqueadaEvent**, **ApartadoVencidoEvent**, **TransferenciaIniciadaEvent** y **DevoluciónAutorizadaEvent**

Mapa de Dominio:



Glosario Actualizado:

Término	Tipo de Concepto	Definición Ubicua en el Dominio
ItemSerie	Aggregate Root / Entidad	Unidad física única (ej. computadora, celular) con un identificador de fábrica (NumeroSerie o IMEI) y ciclo de vida rastreable.
Apartado	Aggregate Root / Entidad	Contrato de reserva comercial mediante un anticipo parcial ($\geq 20\%$) con vigencia de 30 días.
Reserva Lógica	Estado de Dominio	Inhabilitación inmediata de un ItemSerie en la base de datos para impedir su venta a terceros en cualquier sucursal.
NumeroSerie	Value Object	Cadena alfanumérica única o IMEI de 15 dígitos con reglas de formato inmutables que identifica a un ItemSerie.
PorcentajeAnticipo	Value Object	Estructura inmutable que encapsula el monto del enganche y valida que sea igual o mayor al 20% del costo total.

Penalización Vencimiento	Regla de Dominio	Deducción del 10% aplicada al anticipo cuando una orden de apartado supera los 30 días naturales sin saldo liquidado.
Transferencia Inter-Sucursal	Aggregate Root	Proceso de traslado de un ItemSerie entre tiendas que exige confirmación por escaneo en origen y destino.
Autorización Supervisor	Servicio / Permiso	Validación con clave digital requerida para autorizar devoluciones o excepciones fuera de norma.

Ficha de Cada Aggregate:

Apartado

- **Raíz del Agregado (Aggregate Root):** Apartado.
- **Responsabilidades:**
  - Controlar y evaluar el plazo de vigencia de 30 días.
  - Validar que la regla del anticipo mínimo (20%) se cumpla antes de formalizar la orden.
  - Gestionar la lista de abonos del cliente.
  - Calcular la penalización del 10% por cancelación/vencimiento y determinar el saldo a favor.
- **Invariantes (Reglas estrictas de consistencia):**
  - El anticipo inicial **no puede ser menor al 20%** del monto total.
  - No se pueden agregar abonos si el apartado ya fue marcado como Vencido o Liquidado.
  - La suma de los abonos acumulados jamás debe exceder el total del producto.
- **Eventos de Dominio Posibles:** ApartadoCreadoEvent, AbonoRegistradoEvent, ApartadoVencidoEvent, ApartadoLiquidadoEvent.

ItemSerie

- **Raíz del Agregado (Aggregate Root):** ItemSerie.

- **Responsabilidades:**
    - Custodiar la identidad y el estado físico de una unidad individual en la red de 5 sucursales.
    - Controlar y restringir las transiciones de sus estados: Disponible, Reservado, EnTransito, Vendido, EnGarantia.
- **Invariantes (Reglas estrictas de consistencia):**
    - Un ItemSerie en estado Reservado **no puede asignarse a una nueva venta directa**.
    - Un ItemSerie en estado EnTransito no puede ponerse como Disponible en la sucursal destino hasta que se ejecute el escaneo de recepción.
- **Eventos de Dominio Posibles:** SerieBloqueadaEvent, SerieLiberadaEvent, SerieVendidaEvent, SerieEnTransitoEvent.

Venta

- **Raíz del Agregado (Aggregate Root):** Venta.
- **Responsabilidades:**
    - Registrar y procesar la transacción comercial en punto de cobro.
    - Validar la exactitud y cuadratura de los pagos combinados (efectivo, tarjeta, vales).
    - Asociar las series escaneadas con la nota o ticket fiscal emitiendo el comprobante.
- **Invariantes (Reglas estrictas de consistencia):**
    - No se puede cerrar la venta si falta escanear la serie de algún ítem que la requiera.
    - La suma de los métodos de pago ingresados **debe ser exactamente igual al total** de la orden.
- **Eventos de Dominio Posibles:** VentaCompletadaEvent, ComprobanteGeneradoEvent.

Transferencia

- **Raíz del Agregado (Aggregate Root):** Transferencia.
- **Responsabilidades:**
    - Administrar la solicitud, salida, traslado y recepción de mercancía entre tiendas.
- **Invariantes (Reglas estrictas de consistencia):**
    - El número de serie escaneado en la recepción debe coincidir con la guía de salida.
- **Eventos de Dominio Posibles:** TransferenciaIniciadaEvent, TransferenciaCompletadaEvent.

Decisiones Descartadas o Dudosas

Descarte de Sucursal como un Aggregate Pesado:

- Propuesta inicial: Colocar todo el inventario de una tienda dentro de la entidad Sucursal.
- Razón del descarte: Provocaba un cuello de botella de rendimiento y bloqueo transaccional, ya que modificar una sola serie en la Sucursal Centro habría bloqueado las operaciones del resto de los productos de esa tienda.
- Decisión: Se promovió a ItemSerie como su propio Aggregate Root independiente, asociándole únicamente la referencia a la sucursal mediante el Value Object SucursalId.

Ubicación del Cálculo de la Penalización del 10%:

- Propuesta inicial: Calcular la retención del 10% por apartado vencido en la interfaz o en el servicio de aplicación.
- Razón del descarte: Existía el riesgo de que distintos canales (puntos de venta o módulos administrativos) aplicaran porcentajes o fórmulas diferentes.

- Decisión: Encapsular el cálculo de la penalización dentro de la entidad de dominio Apartado para asegurar que la regla se ejecute de manera uniforme sin importar qué interfaz lo invoque.

Etapa 4. Diseñar la Service Layer

Servicio de Aplicación	Método (Caso de Uso)	Parámetros de Entrada	Colaboradores (Interfaces)	Resultado / Salida	Reglas de Negocio que Delega al Dominio
ApartadoApplicationService	CrearApartado	CrearApartadoCommand *(clienteId, sku, numeroSerie, montoAnticipo)*	• IItemSerieRepository • IApartadoRepository • IUnitOfWork	FolioApartadoDTO	• **A ItemSerie:** Validar que el ítem esté en estado Disponible y ejecutar el bloqueo a Reservado. • **A Apartado:** Validar que el anticipo sea ≥ 20% del valor total y fijar el temporizador a 30 días.
ApartadoApplicationService	ProcesarVencimientos	*Ninguno* *(Invocado por Scheduler Nocturno)*	• IApartadoRepository • IItemSerieRepository • IUnitOfWork	ResumenProcesoVencimientoDTO	• **A Apartado:** Evaluar si el plazo transcurrido es > 30 días, calcular la penalización del 10% y generar el saldo a favor del cliente. • **A ItemSerie:** Liberar el ítem reservado a

					estado Disponible.
VentaApplicationService	ProcesarVentaDirecta	ProcesarVentaCommand *(sucursalId, listaSeries, metodosPago)*	• IItemSerieRepository • IVentaRepository • IUnitOfWork	TicketConfirmacionDTO	• **A Venta:** Verificar que la sumatoria de los métodos de pago ingresados iguale exactamente el costo total. • **A ItemSerie:** Validar existencia activa y cambiar el estado de la serie a Vendido.
InventarioApplicationService	SolicitarTransferencia	SolicitarTransferenciaCommand *(sucursalOrigenId, sucursalDestinoId, numeroSerie)*	• IItemSerieRepository • ITransferenciaRepository • IUnitOfWork	FolioTransferenciaDTO	• **A ItemSerie:** Validar que la serie exista físicamente en la sucursal origen y cambiar su estado a EnTránsito. • **A Transferencia:** Crear la guía de traslado entre tiendas.
DevolucionApplicationService	ProcesarDevolucionEspecial	ProcesarDevolucionCommand *(ticketId, numeroSerie,*	• IVentaRepository • IItemSerieRepository	NotaCreditoDTO	• **A PoliticaDevolucionDomainService:** Validar que la antigüedad de la compra sea ≤ 15 días y evaluar las

		claveSupervisor, motivo)	• ISupervisorAuthDomainService • IUnitOfWork		condiciones físicas. • **A ISupervisorAuthDomainService:** Validar la clave digital del Supervisor. • **A ItemSerie:** Reingresar el ítem a estado Disponible.

Etapa 5. Separar persistencia y transacciones

Repository (Interfaz de Dominio)	Necesidad del Dominio	Métodos Propuestos (Firma de Interfaz)	Objeto/Aggregate que Recupera o Guarda
IItemSerieRepository	Rastrear unidades individuales por su número de serie/IMEI, validar disponibilidad y actualizar sus estados.	• FindBySerie(sn: NumeroSerie): ItemSerie • GetDisponiblesBySucursal(sku: SKU, sucursalId: SucursalId): List<ItemSerie> • Save(item: ItemSerie): void	**ItemSerie** (Aggregate Root)
IApartadoRepository	Crear nuevas reservas de inventario, registrar abonos y consultar apartados activos o vencidos.	• FindById(folio: FolioApartado): Apartado • GetVencidos(fechaCorte: DateTime): List<Apartado> • Save(apartado: Apartado): void	**Apartado** (Aggregate Root)
IVentaRepository	Registrar compras completadas en punto de	• FindById(ticketId: TicketId): Venta • Save(venta: Venta): void	**Venta** (Aggregate Root)

	cobro y consultar tickets para trámites de devolución.		
ITransferenciaRepository	Coordinar y monitorear el traslado físico de unidades entre distintas sucursales.	• FindById(transferenciaId: TransferenciaId): Transferencia • GetPendientesBySucursal(sucursalId: SucursalId): List<Transferencia> • Save(transferencia: Transferencia): void	**Transferencia** (Aggregate Root)

Etapa 6. Separar Commands y Queries

Tipo	Nombre	Datos de Entrada	Handler (Manejador)	Resultado / Salida	Justificación
Command	**CrearApartadoCommand**	• clienteId • sku • numeroSerie • montoAnticipo	CrearApartadoCommandHandler	FolioApartadoDTO	Modifica el estado del dominio: ejecuta la reserva lógica del ItemSerie y crea el agregado Apartado aplicando la regla del 20%.
Command	**ProcesarDevolucionCommand**	• ticketId • numeroSerie • claveSupervisor • motivo	ProcesarDevolucionCommandHandler	NotaCreditoDTO	Altera el estado del dominio: reingresa el ItemSerie a estado *Disponible*, genera la nota de crédito y valida las credenciales de Supervisor.
Query	**ConsultarStockGlobalQuery**	• sku • sucursalId *(Opcional)*	ConsultarStockGlobalQueryHandler	StockGlobalReadModelDTO	Consulta de solo lectura que requiere desplegar el stock consolidado de las 5 tiendas en < 3 segundos (CON-01) sin cargar entidades ni agregados pesados.
Query	**ObtenerHistorialSerieQuery**	• numeroSerie	ObtenerHistorialSerieQueryHandler	SerieHistorialReadModelDTO	Consulta la trazabilidad completa del producto (recepción, transferencias,

					ventas o garantías) para atención a clientes sin alterar el dominio.

Etapa 7. Revisar la propuesta con SOLID

Principio	Pregunta de Revisión de la Guía	Evidencia Buscada en la Solución
S	¿Cada clase o servicio tiene una razón principal para cambiar?	Clases extensas, responsabilidades mezcladas (ej. reglas de negocio + envío de notificaciones o formateo de reportes).
O	¿Pueden agregar una variante de regla sin editar una cadena creciente de condiciones?	Condicionales if/else o switch que deben modificarse por cada nueva política de devolución o tipo de garantía.
L	¿Las implementaciones pueden sustituirse sin cambiar el comportamiento esperado?	Subtipos o clases derivadas que rechazan operaciones mediante excepciones o alteran contratos de la superclase.
I	¿Cada consumidor depende solo de las operaciones que utiliza?	Interfaces extensas o monolíticas con métodos irrelevantes para ciertos clientes o componentes.
D	¿La aplicación y el dominio dependen de abstracciones?	Dependencias directas a SQL, frameworks ORM, servicios externos o controladores físicos en la capa de aplicación.

1. Hallazgo sobre el Principio S (Single Responsibility Principle)

- **Evidencia del Caso / U2:** En la propuesta inicial, el servicio ApartadoApplicationService evaluaba los 30 días de vigencia, calculaba la retención del 10% por penalización, actualizaba el stock en la base de datos y redactaba/enviaba directamente el mensaje SMS/correo de notificación al cliente.
- **Impacto Detectado:** Mezcla directa de reglas de negocio, orquestación transaccional e infraestructura de comunicaciones externas. Si el proveedor

de SMS cambiaba su API o formato, se debía modificar la clase que procesa las finanzas del apartado.

- **Cambio y Corrección Aplicada:** Se separaron las responsabilidades. La entidad de dominio Apartado calcula su propia penalización del 10%, el ApartadoApplicationService coordina la actualización del estado, y el envío de notificaciones se delegó a un manejador desacoplado (ApartadoVencidoEventHandler) que escucha el evento ApartadoVencidoEvent.

2. Hallazgo sobre el Principio O (Open/Closed Principle)

- **Evidencia del Caso / U2:** La verificación de devoluciones en punto de venta utilizaba una serie de sentencias if/else anidadas dentro de la clase de servicio para validar si el producto era una computadora, un celular o un accesorio genérico.
- **Impacto Detectado:** Cada vez que TecnoStock incorpore una nueva categoría de productos con reglas especiales de garantía (ej. consolas o componentes que exijan inspección en taller), se tendría que abrir y modificar el código del servicio principal de devoluciones, violando el principio OCP y arriesgando la estabilidad de las devoluciones normales.
- **Cambio y Corrección Aplicada:** Se definió la interfaz IPoliticaDevolucion. Ahora se crean clases independientes como PoliticaDevolucionSerieStandard, PoliticaDevolucionAccesorios o PoliticaDevolucionConSupervisor. El servicio invoca el contrato de la interfaz, permitiendo agregar nuevas políticas extendiendo el código sin modificar los servicios existentes.

3. Hallazgo sobre el Principio L (Liskov Substitution Principle)

- **Evidencia del Caso / U2:** Inicialmente se diseñó una jerarquía donde ItemSinSerie heredaba de la clase ItemSerie. Al intentar ejecutar el método BloquearParaReserva() sobre un ItemSinSerie, la subclase lanzaba una excepción NotSupportedException.

- **Impacto Detectado:** Violación directa del principio de sustitución de Liskov. El código del cliente (como un procesador de ventas o apartados) no podía tratar polimórficamente a las instancias de productos sin romper la ejecución ante excepciones inesperadas.
- **Cambio y Corrección Aplicada:** Se eliminó la relación de herencia incorrecta. Se unificó el concepto del dominio en la entidad ItemSerie únicamente para productos con trazabilidad individual, mientras que los productos masivos sin serie se manejan mediante un control de existencias genérico por lote/SKU sin simular comportamiento de series.

4. Hallazgo sobre el Principio I (Interface Segregation Principle)

- **Evidencia del Caso / U2:** Se había definido una única interfaz de repositorio ITecnoStockRepository que contenía métodos para guardar apartados, consultar ventas, actualizar series, trasladar mercancía y generar reportes administrativos.
- **Impacto Detectado:** Los componentes o servicios especializados (como el manejador de notificaciones de apartados) se veían obligados a depender de una interfaz monolítica con decenas de métodos que no utilizaban. Cualquier cambio en los parámetros de ventas obligaba a recompilar componentes de inventario.
- **Cambio y Corrección Aplicada:** Se fragmentó la interfaz gigante en interfaces pequeñas y orientadas a las necesidades específicas de cada agregado: IApartadoRepository, IItemSerieRepository, IVentaRepository e ITransferenciaRepository.

5. Hallazgo sobre el Principio D (Dependency Inversion Principle)

- **Evidencia del Caso / U2:** En un primer borrador de la capa de aplicación, los servicios instanciaban directamente clases concretas de infraestructura mediante new SqlItemSerieRepository() y hacían llamadas a bibliotecas físicas de lectura de código de barras.

- **Impacto Detectado:** Acoplamiento rígido entre la capa de aplicación y la tecnología subyacente (SQL Server / Lector de Hardware). Esto impedía ejecutar pruebas unitarias automatizadas sin estar conectados a la base de datos real.
- **Cambio y Corrección Aplicada:** Se aplicó inversión de dependencias inyectando abstracciones (IItemSerieRepository, IUnitOfWork, ISupervisorAuthDomainService) a través del constructor de los Application Services. La configuración de las implementaciones concretas de infraestructura se trasladó al contenedor de inyección de dependencias de la aplicación.

Etapa 8. Representar y documentar la arquitectura

Diagrama de Clases DDD

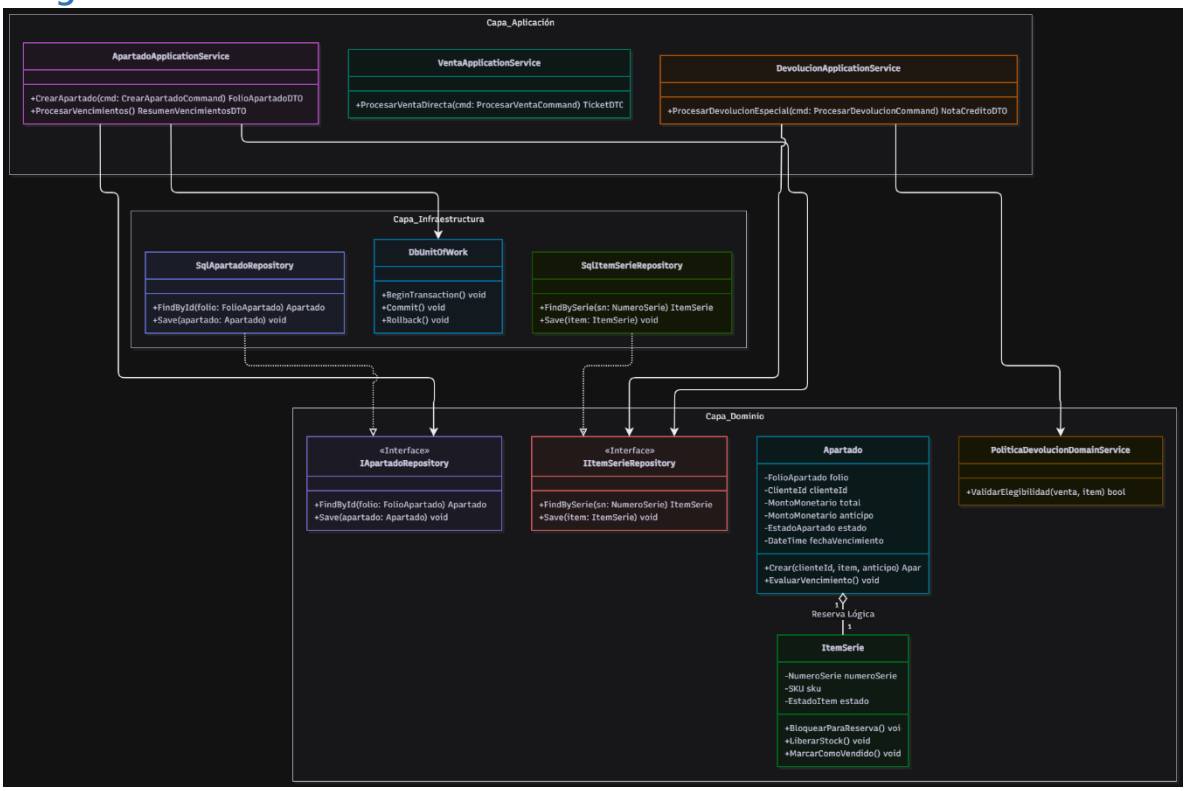


Diagrama de Secuencia 1: Crear Orden de Apartado (Flujo Principal)

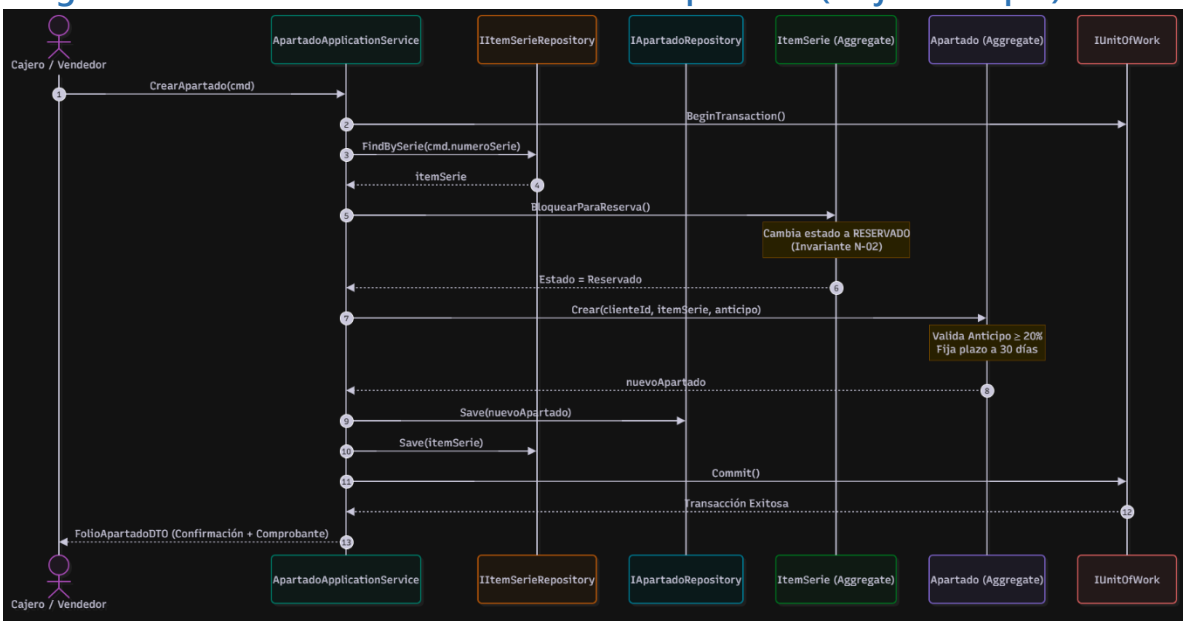
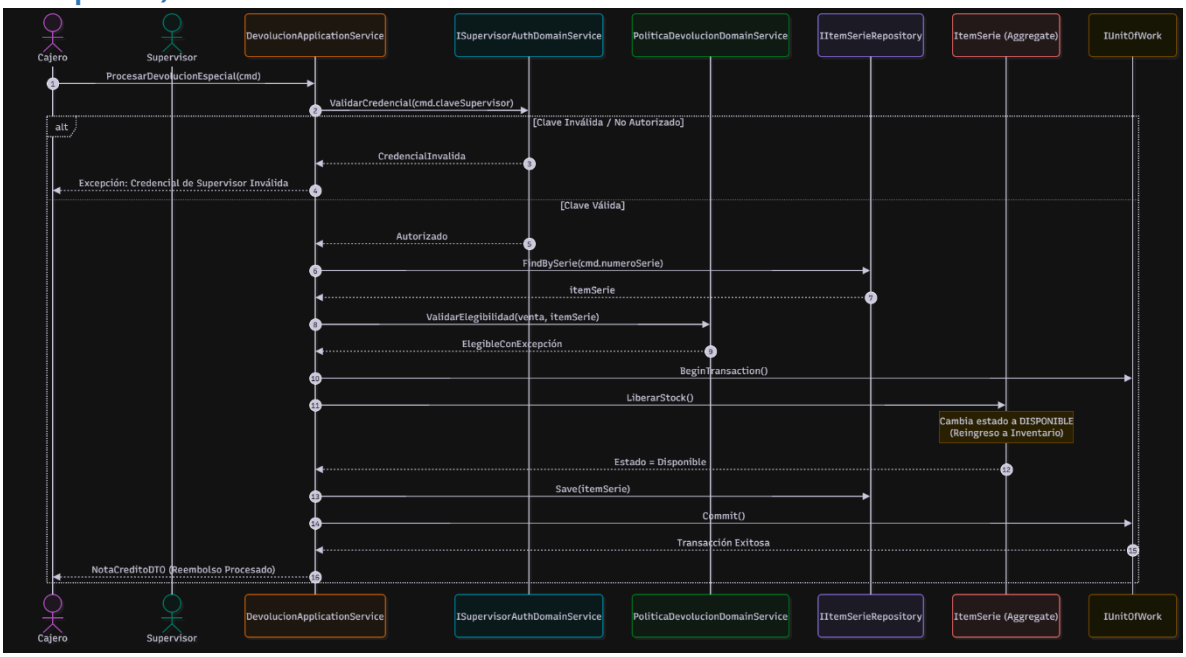


Diagrama de Secuencia 2: Devolución Especial por Supervisor (Flujo Excepción)



Architecture Decision Records

ADR-01: Selección del Patrón Domain Model para la Lógica de Negocio

- **Título:** ADR-01 Selección del Patrón Domain Model.
- **Estatus:** Aceptado.
- **Contexto:** TecnoStock opera 5 sucursales físicas y comercializa electrónicos con trazabilidad por número de serie. La operación requiere garantizar que la mercancía apartada no se venda por error en mostrador, que los anticipos respeten el límite mínimo del 20% y que las devoluciones estén estrictamente validadas. Diseñar la lógica mediante scripts o tablas relacionales generaba riesgo de duplicación de reglas y falta de protección sobre el inventario reservado.
- **Alternativas Consideradas:**
    1. *Transaction Script:* Descartado por el alto riesgo de duplicación de código en terminales de caja y falta de encapsulamiento para proteger el estado del inventario.
    2. *Table Module:* Descartado por su rigidez para representar operaciones orientadas a objetos y coordinar reglas entre múltiples entidades.
    3. *Domain Model:* Seleccionado por encapsular el estado y el comportamiento en objetos de dominio ricos (Aggregates).
- **Decisión:** Adoptar el patrón **Domain Model** como patrón central de arquitectura para la lógica de negocio de TecnoStock.
- **Consecuencias:**
    - *Beneficios:* Encapsulamiento total de invariantes de negocio (ej. la reserva de ItemSerie y la regla del 20% de Apartado se ejecutan de forma segura dentro del dominio), facilidad para realizar pruebas unitarias sobre reglas sin depender de la base de datos.
    - *Costos/Riesgos:* Curva de aprendizaje inicial y mayor cantidad de clases e interfaces en la solución.
- **Trazabilidad:** N-01, N-02, N-03, US-02, US-03, US-04.

ADR-02: Separación de Responsabilidades mediante CQRS Básico

- **Título:** ADR-02 Separación de Commands y Queries (CQRS Básico).
- **Estatus:** Aceptado.
- **Contexto:** La restricción **CON-01** exige que las búsquedas de disponibilidad entre las 5 sucursales respondan en menos de 3 segundos en punto de venta. Cargar el modelo completo de dominio con todas sus entidades e historial para realizar consultas de solo lectura ralentizaba las cajas de cobro.
- **Alternativas Consideradas:**
    1. *Modelo Unificado (Domain Model para Lectura y Escritura):* Descartado por sobrecargar la memoria en cajas y violar el principio SRP.
    2. *CQRS Básico (Separación de Handlers y DTOs de Lectura):* Seleccionado para optimizar las consultas sin alterar la base de datos física.
- **Decisión:** Implementar **CQRS básico** en la capa de aplicación. Las modificaciones de estado se ejecutan mediante *Commands* que pasan por el modelo de dominio, mientras que las consultas utilizan *Queries* que leen directamente de vistas optimizadas (*Read Models*).
- **Consecuencias:**
    - *Beneficios:* Tiempos de respuesta de lectura < 3 segundos en la red de tiendas, DTOs de lectura diseñados a la medida de la pantalla de la caja.
    - *Costos/Riesgos:* Mantenimiento de dos estructuras de clases separadas para lectura y escritura.
- **Trazabilidad:** CON-01, N-01, US-01.

ADR-03: Abstracción de Persistencia con Repository y Unit of Work

- **Título:** ADR-03 Estrategia de Persistencia y Transacciones.
- **Estatus:** Aceptado.
- **Contexto:** El núcleo del dominio de TecnoStock no debe acoplarse a librerías ORM específicas ni a motores SQL. Además, operaciones como registrar un apartado requieren actualizar la entidad Apartado y bloquear el agregado ItemSerie en una misma transacción atómica.
- **Alternativas Consideradas:**
    1. *Invocaciones directas a SQL desde los servicios:* Descartado por violar DIP y acoplar la aplicación a la base de datos.
    2. *Repository Pattern + Unit of Work:* Seleccionado para desacoplar el acceso a datos y garantizar transacciones atómicas.
- **Decisión:** Definir interfaces puras de IRepository en el Dominio e implementarlas en la capa de Infraestructura, utilizando IUnitOfWork para gestionar el inicio, Commit() y Rollback() de transacciones.
- **Consecuencias:**
    - *Beneficios:* Independencia tecnológica total, aislamiento de transacciones ACID y capacidad de simular repositorios en memoria para pruebas.
    - *Costos/Riesgos:* Introducción de interfaces y mappers adicionales entre el modelo relacional y las entidades de dominio.
- **Trazabilidad:** CON-03, N-03, US-02, US-03.

Vibe Coding con IA

Ciclo de Trabajo y Metodología de Validación

Para el diseño de la arquitectura de TecnoStock, el equipo utilizó Inteligencia Artificial Generativa como copiloto de arquitectura para explorar alternativas, generar estructuras iniciales, evaluar violaciones SOLID y sintetizar código de diagramas UML.

El equipo asumió el control y la autoría de cada decisión mediante el siguiente ciclo de 6 pasos:

1. **Preparar contexto:** Se alimentó a la IA con los requerimientos reales del BRD, User Stories, BPMN y Glosario de la Unidad 2.
2. **Solicitar propuesta:** Se solicitaron estructuras concretas exigiendo que la IA declarara abiertamente sus supuestos.
3. **Comparar:** Se contrastaron las respuestas de la IA con la complejidad real de las 5 sucursales y las restricciones de TecnoStock.
4. **Corregir:** Se eliminaron nombres genéricos, clases innecesarias y acoplamientos a base de datos que la IA sugirió por error.
5. **Validar:** Se comprobó la trazabilidad de cada componente contra las necesidades N-01 a N-06.
6. **Registrar:** Se documentó la bitácora de iteraciones mostrando la evidencia de la revisión humana.

Bitacora de IA

Iteración	Objetivo de la Iteración	Prompt Enviado a la IA	Respuesta Útil de la IA	Error o Supuesto Incorrecto Detectado	Decisión y Corrección Humana Aplicada
1	Selección del Patrón de Lógica	*"Con base en estas reglas [N-02: 20% anticipo, 30 días plazo, 10% penalización] y este contexto [5 tiendas retail], compara Transaction Script, Table Module y Domain Model. Declara*	Presentó una matriz comparativa clara identificando los riesgos de duplicidad en Transaction Script.	**Supuesto erróneo:** La IA asumió que TecnoStock era una tienda en línea (E-commerce) con carrito de compras web.	**Se corrigió el contexto:** Se especificó que la venta es física en mostrador de 5 sucursales con lectores de código de barras para números de serie.

		supuestos y muestra consecuencias."			
2	Identificación de Dominio (DDD)	*"Revisa este glosario y reglas [contenido U2]. Propón candidatos a Entidades, Value Objects, Aggregates y Domain Events. Indica evidencia y dudas."*	Identificó correctamente a Apartado e ItemSerie como candidatos a raíces de agregado (*Aggregate Roots*).	**Error de modelado:** La IA sugirió crear un Aggregate gigante llamado Sucursal que contenía todo el inventario de la tienda.	**Se rechazó la estructura:** Agrupar todo el inventario en Sucursal provocaba cuellos de botella SQL. Se convirtió a ItemSerie en Aggregate Root independiente con SucursalId como Value Object.
3	Diseño de Service Layer	*"A partir de estos casos de uso [CU-01 a CU-05], propone firmas de Application Services. Separa coordinación de reglas de dominio y explica cualquier responsabilidad dudosa."*	Generó la orquestación básica con métodos imperativos (CrearApartado, ProcesarVentaDirecta) e inyección de repositorios.	**Violación de capas:** La IA colocó el cálculo de la penalización del 10% por apartado vencido dentro del Application Service.	**Se movió la regla:** Se extrajo el cálculo financiero de la Service Layer y se encapsuló dentro del método EvaluarVencimiento() de la entidad de dominio Apartado.
4	Revisión de Principios SOLID	*"Analiza este diseño de clases y señala posibles violaciones SOLID en el manejo de*	Detectó la violación de SRP al tener un servicio enviando SMS/Emails y actualizando la base de datos	**Solución sobre-diseñada:** La IA propuso implementar un bus de eventos	**Se simplificó:** Dado el alcance del proyecto, se rechazó la infraestructura distribuida y se utilizó un manejador de eventos en

		devoluciones y notificaciones. No reescribas todo; identifica evidencia y alternativas."	simultáneamente.	distribuido con RabbitMQ/Kafka para notificar los vencimientos.	memoria (InProcessDomainEventHandler).
5	Generación de Diagramas UML	*"Convierte esta arquitectura validada [clases y secuencia de apartado/devolución] en un borrador PlantUML/Mermaid. No agregues clases o relaciones no descritas."*	Produjo el código fuente listo para renderizar en Mermaid/Draw.io sin errores de sintaxis.	**Nombres incoherentes:** La IA utilizó nombres en inglés (ItemSerialEntity, LayawayOrder) ignorando el glosario en español.	**Alineación con Lenguaje Ubicuo:** Se renombraron todas las clases y métodos en el código fuente para reflejar el glosario oficial (ItemSerie, Apartado, FolioApartado).

Etapa 9. Validar la Propuesta

Lista de comprobación y validación

A continuación, se presentan los resultados de la verificación técnica realizada sobre la propuesta final:

Criterio de Validación	Pregunta de Evaluación de la Guía	Resultado del Equipo	Evidencia en el Diseño
Trazabilidad	¿Cada componente importante responde a una necesidad, regla o caso de uso?	Validado al 100%.	Todos los Aggregates, Application Services y DTOs están vinculados a necesidades N-01 a N-06.

Lenguaje Ubicuo	¿Los mismos términos significan lo mismo en BRD, UML, código y ADR?	Validado al 100%.	Se estandarizaron los términos ItemSerie, Apartado, Reserva Lógica, Transferencia y Supervisor.
Responsabilidad	¿Las reglas permanecen en el dominio o en el patrón elegido con una justificación?	Validado.	Las reglas de negocio (20% anticipo, 30 días, 10% penalización) están encapsuladas en las entidades de dominio.
Consistencia	¿Los diagramas de clase y secuencia representan la misma arquitectura?	Validado.	Los participantes de los diagramas de secuencia existen en el diagrama de clases DDD con las mismas firmas.
Persistencia	¿El dominio evita depender directamente de SQL, ORM o detalles de infraestructura?	Validado.	El dominio utiliza interfaces de repositorio (IRepository) sin referencias a SQL ni librerías de ORM.
Transacciones	¿El límite de la operación y su manejo de error están definidos?	Validado.	Se implementó el patrón IUnitOfWork que delimita el Commit() y Rollback() por caso de uso.
Uso de IA	¿Cada aporte de IA fue revisado y su uso está registrado?	Validado.	Se incluyó la bitácora de 5 iteraciones documentando errores detectados y correcciones humanas.

Matriz de trazabilidad

Esta matriz demuestra cómo cada elemento identificado en la Unidad 2 (BRD, User Stories, BPMN) se satisface mediante un componente específico de la arquitectura diseñada en la Unidad 3:

Necesidad BRD (U2)	Historia de Usuario / Gherkin (U2)	Caso de Uso (U3)	Componente de Dominio / Aplicación (U3)	Decisión Arquitectural / ADR
N-01: Integración global de stock.	**US-01:** Consulta de inventario global.	**CU-01:** Venta Directa / Consulta.	ConsultarStockGlobalQueryHandler, StockGlobalReadModelDTO.	**ADR-02:** CQRS básico para lecturas rápidas.
N-02: Bloqueo automático por apartado.	**US-03 / US-04:** Anticipo 20% y Reserva Lógica.	**CU-02 / CU-03:** Crear y Vencer Apartado.	Aggregate Root Apartado, ItemSerie.BloquearParaReserva().	**ADR-01:** Pattern Domain Model para invariantes.
N-03: Identificación por número de serie.	**US-02 / US-11:** Captura e ingreso de serie.	**CU-01:** Venta con Serie.	Aggregate Root ItemSerie, Value Object NumeroSerie.	**ADR-01:** Ciclo de vida y estados de serie.
N-04: Validaciones para devoluciones.	**US-09 / US-10:** Devolución con	**CU-05:** Devolución Especial.	PoliticaDevolucionDomainService, ISupervisorAuthDomainService.	**Revisión SOLID:** Principio OCP para políticas.

	Supervisor.			
N-05: Flujo inter-sucursal de mercancía.	**US-07 / US-08:** Solicitud y despacho de traslado.	**CU-04:** Transferencia Inter-sucursal.	Aggregate Root Transferencia, InventarioApplicationService.	**ADR-03:** Transacción atómica vía Unit of Work.
CON-01: Respuesta de stock < 3 **segundos.**	**US-01:** Despliegue inmediato de stock.	**CU-01:** Consulta Global.	Handler de Query desacoplado con SQL/Dapper directo.	**ADR-02:** CQRS sin instanciar agregados.

Enlace de repositorio en GitHub: https://github.com/Angel-Jmz/TecnoStock.git